PagedAttention in MiniTorch

Rigorous Memory, Capacity, Prefix-Reuse, and Branching Results for Block-Granular KV Storage

NingRui Yang **Yixiang Dai**

Carnegie Mellon University — Electrical & Computer Engineering — 11-868 Large Language Model Systems

CUDA Kernel · Prefix Cache · Capacity Curves · Branch Sharing

The updated codebase evaluates realistic reservation waste, empirical capacity, prefix reuse, and branching-memory savings.

96.6%	2.00x	1.90x	78%	79%
KV Efficiency	**Capacity Gain**	**Prefix Prefill Speedup**	**Parallel KV Savings**	**Beam KV Savings**
reserved vs. live KV on realistic mix	vs. `seq_len+32` static cap	at 75% shared prompt	shared fork vs. naive clone	winner-takes-all trunk sharing

Why This Matters

KV reservation, not attention math, dominates serving efficiency. Static allocation pays for a worst-case cap per request:

$$\mathrm{KV}_{\mathrm{static}} = 2 \cdot L \cdot H \cdot d_h \cdot N_{\mathrm{cap}} \cdot B$$

- Reserved bytes that never get used directly cap concurrency.
- Static waste grows when prompt lengths are non-aligned with the chosen block or cap size.
- PagedAttention replaces slabs with reusable blocks and copy-on-write only where branches diverge.

Problem Statement

LLM serving is bottlenecked not by attention math but by **KV-cache memory**. Static contiguous allocation reserves a worst-case slab per request, so most KV bytes are reserved but never used.

Goals

- Quantify reserved-vs-used KV waste under a realistic non-aligned workload.
- Measure capacity gain, prefix-prefill speedup, and branching-memory savings of paged allocation in MiniTorch + CUDA.
- Separate allocator-focused synthetic measurements from claims about end-to-end serving latency.

Methodology: What We Built

MiniTorch implementation, CUDA decode kernel, and a benchmark harness isolate where paged KV helps.

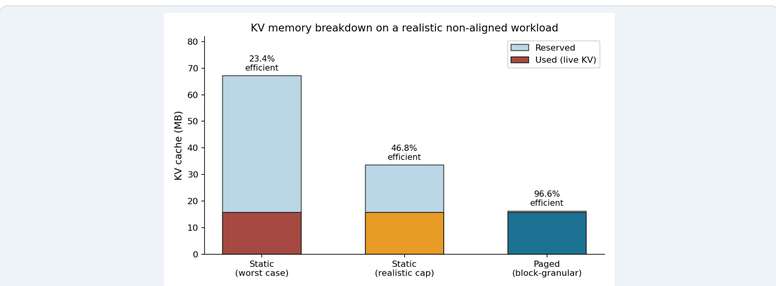
Allocator + sharing

- Global per-layer `key_cache` / `value_cache` with shape `(num_blocks, block_size, n_head, head_dim)`.
- `BlockTable` maps logical tokens to blocks; full-block hash chains drive prefix reuse.
- Requests hold block mappings rather than private contiguous slabs.
- `fork_sequence()` and `clone_sequence()` contrast shared vs. duplicated branching state.

Rigorous benchmark suite

- `run_rigorous_benchmark.py`: six experiments covering memory, capacity, prefix prefill, and branching.
- Realistic non-aligned workload, allocate-until-OOM capacity, warmup + timed trials.
- Each result isolates allocator policy, so memory savings and reuse effects are visible before scheduler noise dominates.

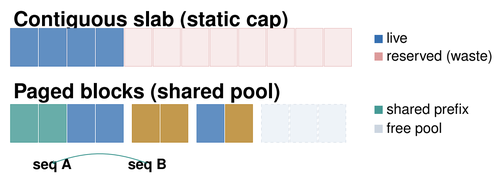
Result 1: KV Memory Breakdown



On a non-aligned mix, paged allocation reaches **96.6%** efficiency vs. **46.8%** for a `seq_len+margin` cap and **23.4%** for worst-case static.

How PagedAttention Works

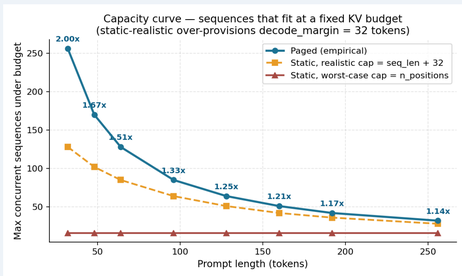
Contiguous vs. paged allocation



Inference flow

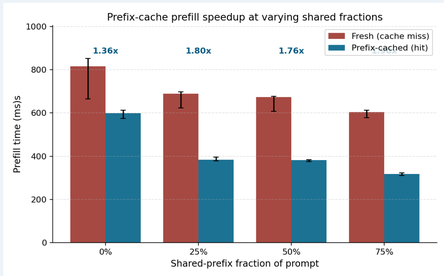
1. **Prefill:** `BlockTable` maps logical tokens to physical blocks; K/V written into the pool.
2. **Prefix lookup:** hash chain finds the longest reusable prefix; only the suffix runs.
3. **Decode / branch:** append to the tail, or fork and pay copy-on-write only on divergence.
4. **Retire:** blocks return to the pool when ref counts hit zero.

Result 2: Capacity Under Fixed Budget



Under the same KV budget, paged allocation fits up to **2.00x** more concurrent sequences (short prompts) and still **1.14x** more at 256 tokens.

Result 3: Prefix-Cache Prefill



Shared-prefix fractions **0/25/50/75%** yield **1.36x** / **1.80x** / **1.76x** / **1.90x** prefill speedup. Suffixes rotate across trials so gains track real cache hits.

Measurement Notes

- Figs. 1, 2, 5, 6 are allocator-backed measurements; Fig. 4 is timed (1 warmup + 3 trials).
- Model: `n_embd=512`, `n_head=8`, `n_layers=4`, `block_size=16`, `num_kv_blocks=512`.
- Workload mix `[48,67,112,160,256,80,35,200]`; capacity sweep `32..256`; prefix sweep `0/.25/.5/.75`; branching prompts `32/67/128`.
- Synthetic micro-benchmarks isolate allocator behavior; end-to-end latency is future work.

Validation

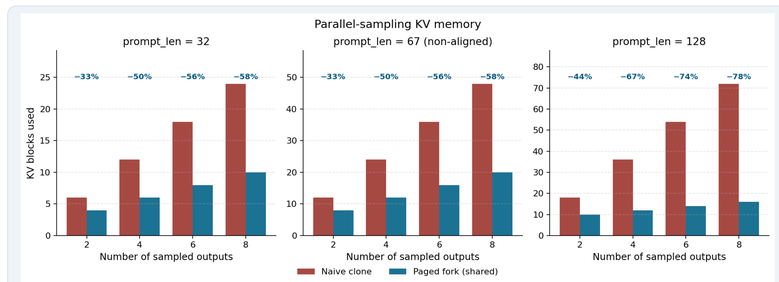
- **33 passing tests**; CUDA kernel agrees with the Python reference within **1e-5**.
- Prefill/decode parity verified against shared Q/K/V weights.

Key Takeaways

One allocator mechanism improves multiple serving bottlenecks at once:

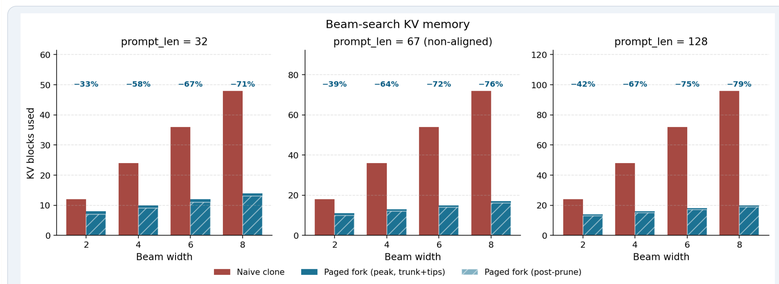
- **Memory: 96.6%** KV efficiency on a realistic mix.
- **Capacity:** up to **2.00x** more concurrent sequences.
- **Prefix reuse:** up to **1.90x** faster prefill at 75% shared.
- **Branch sharing: 58–79%** KV cut for sampling and beams.
- **Implementation setting:** these gains come from a MiniTorch + CUDA prototype, not a paper-only model.

Result 4: Parallel-Sampling Memory



Forks share the prompt trunk and allocate only divergent decode blocks: **33–78%** KV savings, biggest with many outputs from a long shared prompt.

Result 5: Beam-Search Memory



Beams share prompt and surviving trunk: peak KV savings **79%** at beam width 8, with even less memory after pruning.

Serving Implications

- Better KV efficiency directly increases admission headroom under a fixed memory budget.
- Prefix and trunk sharing matter most for multi-output workloads, where many continuations reuse the same prompt.
- End-to-end gains still depend on batching, scheduling, and kernel quality, but allocator policy sets the memory ceiling first.

Final Takeaway

Block-granular KV turns worst-case reservation into **on-demand allocation and sharing** — delivering more capacity, faster cached prefill, and dramatically cheaper branching from one mechanism. The systems lesson is that better KV residency policy improves throughput, reuse, and branching efficiency without changing the model itself.

Acknowledgments

We thank the **11-868 LLM Systems** course staff at Carnegie Mellon University for guidance, and the **MiniTorch** project and the original **vLLM** / **PagedAttention** work (Kwon et al., SOSP 2023) for the framework and design we built upon.